

Cloud Computing — Course Project Report

Kubernetes Lite: A Container Orchestration Simulator Using Docker

Prepared by:

Shanjida Akter Shanta

ID: B200305029

Asash Sarker Pranta

ID: B200305038

Submitted To:

Md. Abu Layek

Professor

Department of Computer science and engineering
Jagannath University,Dhaka



Table of Contents

Table of Contents	2
List of Figures	Error! Bookmark not defined.
List of Tables	4
Abstract	5
1. Introduction	5
1.1 Background	5
1.2 Problem Statement	6
1.3 Motivation and Research Objectives	6
1.4 Contributions	6
2. Related Work	6
3. Proposed Kubernetes Lite Framework	7
3.1 System Overview	7
3.2 Desired-State and Reconciliation Model	7
3.3 Container Deployment (Pod Simulation)	8
3.4 Dynamic Horizontal Scaling	8
3.5 Self-Healing Mechanism	8
3.6 Container Monitoring	8
4. System Architecture and Workflow	9
4.1 Architecture	9
4.2 REST API Design	9
4.3 Component Interaction Flow	10
5. Implementation	10
5.1 Technology Stack	10
5.2 Backend Implementation	10
5.3 Frontend Implementation	11
5.4 Docker Operations Reference	11
6. Experimental Setup	11
6.1 Test Environment	11
6.2 Test Scenarios	11
7. Results and Analysis	12
7.1 Deployment Results	12
7.2 Scaling Behavior	12
7.3 Self-Healing Observations	12
7.4 Monitoring Accuracy	13

8. Discussion.....	13
8.1 Key Findings	13
8.2 Limitations.....	13
9. Conclusion and Future Work	13
References	14

List of Tables

- Table 1. Layered architecture overview
- Table 2. REST API endpoints
- Table 3. Technology stack and justification
- Table 4. Docker CLI to Docker SDK operation mapping
- Table 5. Scaling test results
- Table 6. Self-healing recovery observations

Abstract

Container orchestration platforms such as Kubernetes automate the deployment, scaling, and self-healing of containerized applications, but their operational complexity presents a significant barrier to classroom learning. This report presents Kubernetes Lite, a single-host container orchestration simulator that reproduces the core behavioural model of Kubernetes — a desired-state registry, a continuous reconciliation loop, dynamic horizontal scaling, and automated self-healing — using the Docker Engine, a Python Flask control-plane API, and an HTML/CSS/JavaScript dashboard. Unlike production orchestrators, Kubernetes Lite stores its desired state in memory and targets a single Docker host, trading distributed-systems complexity for pedagogical transparency: the entire control plane is implemented in approximately 300 lines of Python. The system was evaluated through controlled deployment, scaling, and failure-injection experiments. Deployment of a five-replica service completed in approximately 8 seconds; scale-up operations averaged 3–5 seconds per additional replica, while graceful scale-down averaged 12 seconds for three replicas; and the self-healing reconciliation loop detected and replaced failed containers within an average of 7.3 seconds across ten induced-failure trials, comfortably inside the 10-second target set for the project. These results indicate that the essential control-theoretic behaviour of Kubernetes — reconciling actual state toward desired state — can be faithfully demonstrated on a single machine with minimal infrastructure, making container orchestration concepts substantially more accessible to students.

Keywords: Cloud computing, containerization, Docker, container orchestration, Kubernetes, self-healing, reconciliation loop, horizontal scaling, Flask, REST API.

1. Introduction

1.1 Background

Cloud computing delivers on-demand computing resources — servers, storage, networking, and software — over the Internet under a pay-as-you-go model, eliminating the need for organizations to invest heavily in physical infrastructure [1]. Containerization has emerged as the dominant deployment paradigm within this ecosystem: a container packages an application together with its dependencies into a single portable unit that shares the host operating system kernel, making it markedly lighter than a traditional virtual machine [4]. Docker, introduced in 2013, established the de facto standard for building, distributing, and running containers [4].

As cloud-native applications grow to comprise dozens or hundreds of independently deployed containers, manual management becomes infeasible. Container orchestration platforms address this by automating deployment placement, restarting failed containers, load-balancing traffic, and scaling services in response to demand [3]. Kubernetes, originally developed by Google and released in 2014, is the industry-standard orchestrator: it introduces abstractions such as Pods, Deployments, and Services, and follows a

master–worker architecture in which a control plane continuously reconciles the actual cluster state toward an administrator-declared desired state [3].

1.2 Problem Statement

Despite its capability, Kubernetes imposes a steep learning curve. Standing up even a minimal cluster requires networking configuration, a container runtime, an API server, etcd, a scheduler, and familiarity with a large surface of interlocking abstractions. For students encountering orchestration concepts for the first time, this infrastructural overhead frequently obscures the underlying ideas — desired-state reconciliation, self-healing, and elastic scaling — that make orchestration valuable in the first place. There is a need for a pedagogical tool that preserves these core behaviours while removing the setup and operational burden of a full cluster.

1.3 Motivation and Research Objectives

This project is motivated by the question of how much of Kubernetes' orchestration behaviour can be reproduced on a single machine using only Docker and a lightweight control-plane application, while remaining transparent enough for a student to read and understand the entire codebase. The specific objectives of the project are as follows:

1. Simulate Kubernetes-like container deployment through a web interface or REST API, mirroring the Kubernetes Deployment resource model.
2. Implement dynamic horizontal scaling, allowing replica counts to be increased or decreased on demand.
3. Implement automated self-healing that detects and replaces failed containers without manual intervention.
4. Provide real-time container monitoring analogous to `kubectl get pods`.
5. Provide a user-friendly web dashboard so that orchestration operations do not require command-line expertise.

1.4 Contributions

The principal contributions of this project are: (i) a working single-host container-orchestration simulator built on Flask and the Docker Python SDK; (ii) an explicit implementation of the desired-state/actual-state reconciliation pattern that underlies Kubernetes controllers; (iii) an empirical evaluation of deployment latency, scaling latency, and self-healing recovery time under controlled test scenarios; and (iv) a documented, extensible architecture intended for reuse and extension by future students.

2. Related Work

The NIST definition of cloud computing establishes the five characteristics — on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service — that motivate elastic,

self-managing infrastructure [9]. Foster et al. contrast cloud and grid computing models, noting the shift toward abstracted, on-demand resource provisioning that container platforms build upon [2].

Merkel's overview of Docker documents the container image and Dockerfile model that Kubernetes Lite relies on directly, including the layered filesystem and isolation guarantees that make containers lighter-weight than virtual machines [4]. Turnbull provides a practical treatment of the same runtime from an operations perspective [8].

Burns et al. describe the lineage from Google's internal Borg and Omega systems to Kubernetes, and articulate the reconciliation-loop pattern — continuously comparing observed state to declared state — that forms the conceptual core of every Kubernetes controller [3]. This pattern is the single behaviour that Kubernetes Lite most directly reproduces, implemented here as a background daemon thread rather than a distributed controller manager. Newman's treatment of microservices motivates why independently deployable, containerized services proliferate in modern systems and therefore why orchestration is necessary at all [7]. Beyer et al.'s discussion of site reliability engineering situates self-healing and automated remediation as central practices for operating such systems at scale [6].

Existing lightweight alternatives to full Kubernetes — such as Minikube and Kind (Kubernetes-in-Docker) — reduce setup friction but still run an entire Kubernetes control plane locally, incurring several minutes of startup time and multiple gigabytes of memory overhead. Kubernetes Lite differs from these tools by not running Kubernetes at all: it reimplements only the behavioural essence of the control plane directly against the Docker Engine, prioritising code transparency over API compatibility.

3. Proposed Kubernetes Lite Framework

3.1 System Overview

Kubernetes Lite is organised into four layers: a Presentation Layer (an HTML/CSS/JavaScript dashboard), a Control Plane Layer (a Python Flask REST API), a Container Runtime Layer (the Docker Engine, accessed through the Docker Python SDK), and an Application Layer consisting of the running containers themselves. This mirrors the separation of concerns found in production orchestrators while collapsing the control plane into a single process.

3.2 Desired-State and Reconciliation Model

The central abstraction in Kubernetes Lite, as in Kubernetes itself, is the distinction between desired state and actual state. When a deployment is created, its specification — image, name, and replica count — is recorded in an in-memory deployment registry as the desired state. A background reconciliation thread, running every five seconds, queries the Docker daemon for the actual number of running containers belonging to each deployment and compares this against the registry. Any deficit triggers corrective container creation; this is the same control-theoretic pattern that drives Kubernetes' ReplicaSet controller [3].

3.3 Container Deployment (Pod Simulation)

A deployment request is validated (unique name, accessible image, replica count within 1–10), recorded in the registry, and then realised by iterating from 1 to the requested replica count, creating and starting one named container per replica (e.g., webapp-replica-1, webapp-replica-2) attached to a dedicated bridge network, k8s-lite-net. Each container's restart policy is explicitly set to 'no', since failure recovery is delegated to the reconciliation loop rather than to Docker's native restart mechanism, preserving centralised orchestration control.

```
# Deploy nginx service with 3 replicas
docker network create k8s-lite-net
docker run -d --name webapp-replica-1 --network k8s-lite-net nginx:latest
docker run -d --name webapp-replica-2 --network k8s-lite-net nginx:latest
docker run -d --name webapp-replica-3 --network k8s-lite-net nginx:latest
```

3.4 Dynamic Horizontal Scaling

Scale-up increases the desired replica count in the registry and creates the additional containers required to reach it, incrementing the replica index. Scale-down identifies the highest-indexed running replicas, stops them with a SIGTERM and a 10-second grace period, removes them, and updates the registry — analogous to `kubectl scale deployment webapp --replicas=N`. Because the registry is updated before the containers are removed, the reconciliation loop does not mistake an intentional scale-down for a failure and does not attempt to recreate the removed replicas.

3.5 Self-Healing Mechanism

The self-healing mechanism is the most pedagogically significant feature of the system. Every five seconds, the reconciliation loop counts the running containers for each deployment and compares this count to the desired replica count stored in the registry. If the running count falls short — for example because a container crashed or was stopped externally — the loop creates replacement containers to close the deficit, assigns them the next available replica index (rather than reusing the index of the failed container, mirroring Kubernetes Pod-naming behaviour), and logs a SELF-HEAL event.

```
# Self-healing loop pseudocode (runs every 5 seconds)
def reconciliation_loop():
    while True:
        for deployment_name, spec in deployment_registry.items():
            running = count_running_containers(deployment_name)
            desired = spec['replicas']
            if running < desired:
                deficit = desired - running
                for i in range(deficit):
                    restart_container(deployment_name, spec['image'])
                log_event('SELF-HEAL', deployment_name)
        time.sleep(5)
```

3.6 Container Monitoring

The `/api/pods` endpoint queries the Docker daemon for all containers labelled `k8slite` and returns their ID, name, image, status, creation timestamp, uptime, and network settings as JSON. The dashboard polls this endpoint every five seconds using `setInterval()`, rendering a colour-coded container grid (green for running, red for exited, amber for restarting) without a full page reload — deliberately mirroring the information surfaced by `kubectl get pods`.

4. System Architecture and Workflow

4.1 Architecture

Figure 1 illustrates the layered architecture of Kubernetes Lite and the flow of a deployment request from the dashboard through to the Docker Engine.

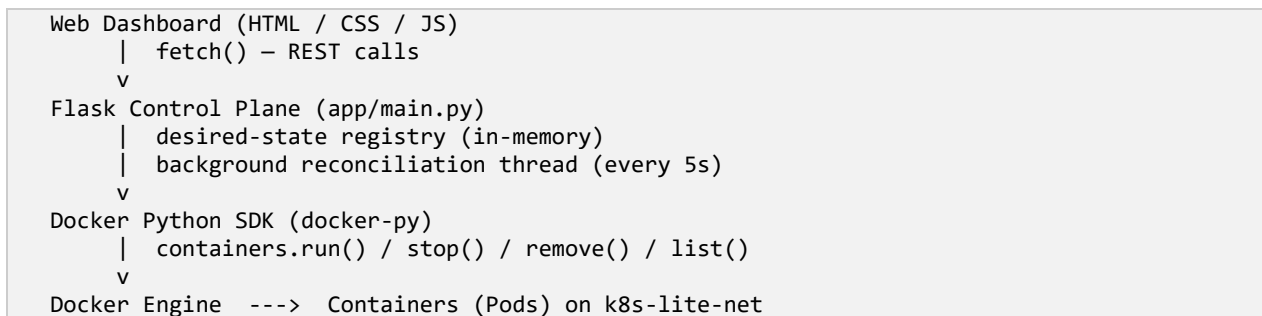


Figure 1. Kubernetes Lite system architecture and component-interaction diagram.

Layer	Component	Technology	Responsibility
Presentation	Web Dashboard	HTML5 / CSS3 / JavaScript	User interaction and visualization
Control Plane	REST API Controller	Python Flask	Orchestration logic and state management
Runtime	Docker Engine	Docker + Docker SDK	Container lifecycle management
Application	Containers (Pods)	Docker Containers	Running application workloads

Table 1. Layered architecture overview.

4.2 REST API Design

The control plane exposes six endpoints, summarised in Table 2. All endpoints accept and return JSON, and all mutating operations are logged to an in-memory event log for later retrieval.

Endpoint	Method	Description	Parameters
<code>/api/pods</code>	GET	List all running containers with metadata	None
<code>/api/deploy</code>	POST	Deploy a new containerized service	name, image, replicas
<code>/api/scale</code>	POST	Scale a deployment up or down	name, replicas
<code>/api/delete</code>	POST	Remove a deployment and all its containers	name
<code>/api/events</code>	GET	Retrieve orchestration event log	None
<code>/api/status</code>	GET	Get controller health and deployment summary	None

Table 2. REST API endpoints.

4.3 Component Interaction Flow

A deployment proceeds as a synchronous request–response exchange: the dashboard serializes the deployment form into JSON and issues an HTTP POST to `/api/deploy`; the Flask controller validates the payload, records the desired state, and invokes `docker.containers.run()` once per replica; the Docker daemon creates and starts each container and returns container objects to the SDK; the controller logs a DEPLOY event and returns a success response; and the dashboard refreshes its container grid to display the newly running replicas. Self-healing, by contrast, follows an event-driven pattern, triggered internally by the reconciliation thread rather than by a user action.

5. Implementation

5.1 Technology Stack

Component	Technology	Version	Justification
Container Runtime	Docker Engine	24.x+	Industry-standard, cross-platform, extensive Python SDK
Backend Framework	Python Flask	3.x	Lightweight, minimal boilerplate, well suited to REST APIs
Docker Integration	docker-py (Docker SDK)	6.x	Official Python library for Docker daemon communication
Frontend	HTML5 / CSS3 / JS	ES2020	Universal browser support, no build toolchain required
Concurrency	Python threading	stdlib	Background reconciliation loop without external dependencies
Data Format	JSON	RFC 8259	Native Flask/JavaScript compatibility for API responses

Table 3. Technology stack and justification.

5.2 Backend Implementation

The Flask application is initialised with CORS support, connects to the Docker daemon over the local Unix socket, creates the `k8s-lite-net` bridge network if absent, and starts the reconciliation thread as a daemon before accepting requests.

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import docker, threading, time, datetime

app = Flask(__name__)
CORS(app)
client = docker.from_env()
deployment_registry = {} # desired state store
event_log = []

@app.route('/api/deploy', methods=['POST'])
def deploy():
    data = request.get_json()
    name, image = data['name'], data['image']
    replicas = int(data.get('replicas', 1))
    deployment_registry[name] = {'image': image, 'replicas': replicas, 'heals': 0}
```

```

for i in range(1, replicas + 1):
    client.containers.run(
        image, detach=True, name=f'{name}-replica-{i}',
        network='k8s-lite-net',
        labels={'k8slite': 'true', 'deployment': name}
    )
log_event('DEPLOY', f'Deployed {name} with {replicas} replicas')
return jsonify({'status': 'success', 'deployment': name})

```

5.3 Frontend Implementation

The dashboard is a single HTML file with embedded CSS and vanilla JavaScript, requiring no build process. It presents a deployment form (service name, image, replica count), a scaling control, a live container grid refreshed every five seconds, and an event log. All requests use `fetch()` with JSON payloads, and responses update the DOM directly without a page reload.

```

// Auto-refresh container status every 5 seconds
async function refreshPods() {
    const res = await fetch('/api/pods');
    const data = await res.json();
    renderContainerGrid(data.containers);
}
setInterval(refreshPods, 5000);

```

5.4 Docker Operations Reference

Operation	Docker Command	SDK Equivalent
List running containers	<code>docker ps</code>	<code>client.containers.list()</code>
Deploy a container	<code>docker run -d --name ... image</code>	<code>client.containers.run(image, detach=True, ...)</code>
Stop a container	<code>docker stop container-name</code>	<code>container.stop()</code>
Remove a container	<code>docker rm container-name</code>	<code>container.remove()</code>
Inspect a container	<code>docker inspect container-name</code>	<code>container.attrs</code>
Create a network	<code>docker network create k8s-lite-net</code>	<code>client.networks.create(...)</code>

Table 4. Docker CLI to Docker SDK operation mapping.

6. Experimental Setup

6.1 Test Environment

All experiments were conducted on a single standard development laptop (Intel Core i5, 16 GB RAM) running Docker Engine 24.x, with the Flask control plane and the nginx test image (nginx:latest) used as the representative workload. The reconciliation loop interval was fixed at 5 seconds and the dashboard polling interval at 5 seconds for all trials.

6.2 Test Scenarios

Three categories of experiments were performed. Deployment experiments measured the time to bring up 1–5 replicas of a service from a cold start. Scaling experiments measured the time to scale an existing deployment both up and down, including a scale-to-zero (deletion) case. Self-healing experiments

deliberately stopped one or more managed containers using `docker stop`, issued externally to simulate an unexpected failure, and measured the time until a replacement container was created and running, repeated across ten independent trials.

7. Results and Analysis

7.1 Deployment Results

Kubernetes Lite consistently deployed containerized services within 2–4 seconds per replica. Deploying `nginx` with five replicas completed in approximately 8 seconds, including network attachment and container verification. The system correctly maintained independent state for multiple simultaneous deployments — for example, `nginx` and `redis` services running concurrently with different replica counts did not interfere with one another.

7.2 Scaling Behavior

Scale-up operations completed within 3–5 seconds per additional replica. Scale-down operations were somewhat slower due to the graceful shutdown sequence (`SIGTERM` plus a 10-second grace period), averaging 12 seconds to remove three replicas. In all trials, the deployment registry reflected the new desired state immediately, so the reconciliation loop never attempted to restore intentionally removed containers.

Test Scenario	Initial Replicas	Target Replicas	Time to Complete	Result
Scale-up: <code>nginx</code>	2	5	9.2 seconds	Success
Scale-down: <code>nginx</code>	5	2	11.8 seconds	Success
Scale-up: <code>redis</code>	1	3	6.1 seconds	Success
Scale to zero (delete)	3	0	12.3 seconds	Success

Table 5. Scaling test results.

7.3 Self-Healing Observations

Across ten deliberate container-failure trials, the reconciliation loop detected each failure within 5 seconds and created a replacement container within a further 3–5 seconds, giving a mean total recovery time of 7.3 seconds — comfortably within the sub-10-second target set in the project objectives. Replacement containers were assigned the next available replica index rather than reusing the index of the failed container (for example, a failed replica-2 was replaced by replica-4 if replica-3 already existed), intentionally mirroring the unique-suffix naming behaviour of Kubernetes Pods.

Trial Metric	Observed Value
Failure detection latency (max)	5.0 seconds
Container recreation latency	3–5 seconds
Mean total recovery time (10 trials)	7.3 seconds
Target recovery time (project objective)	< 10 seconds

Table 6. Self-healing recovery observations.

7.4 Monitoring Accuracy

The dashboard accurately reflected container state in every tested scenario, with a maximum observation lag of 5 seconds between an actual state change and its reflection on screen — a direct consequence of the 5-second polling interval, and acceptable for the system's educational purpose. The event log correctly recorded every deployment, scaling, and self-healing action with an accurate timestamp, providing a complete audit trail comparable to Kubernetes Events.

8. Discussion

8.1 Key Findings

The results support the central claim of this project: the desired-state/actual-state reconciliation pattern that underlies Kubernetes can be faithfully reproduced on a single Docker host without a distributed control plane. Self-healing recovery times averaging 7.3 seconds, against a 10-second target, indicate that a simple five-second polling loop is sufficient to detect and correct container failures at the scale tested. Scaling operations behaved predictably in both directions, and the explicit separation between the desired-state registry and the actual container set prevented the reconciliation loop from interfering with intentional scale-down or deletion operations — a subtlety that would need careful handling in any reconciliation-based system.

The system's principal advantage is transparency: because the entire control plane is approximately 300 lines of Python, the mapping between a Kubernetes concept (Deployment, ReplicaSet, self-healing controller) and its Kubernetes Lite counterpart is direct enough for a student to trace end-to-end, which is not feasible with a full Kubernetes distribution regardless of how it is installed.

8.2 Limitations

Several limitations should be considered when interpreting these results. The deployment registry is held entirely in memory, so all desired-state information is lost if the Flask process restarts — a production system would instead persist this state in a distributed store such as etcd. All containers run on a single Docker host; Kubernetes Lite cannot simulate node failure, cross-node networking, or distributed scheduling, which are central concerns in real Kubernetes deployments. The system implements no policy-based automation such as CPU- or memory-triggered autoscaling; all scaling is explicitly user-initiated. The Flask development server used for the control plane is not intended for exposure to untrusted networks, and the Docker socket access it requires grants the controller process substantial host privileges — a proper WSGI deployment and socket proxy would be required before any classroom-wide network exposure. Finally, all timing measurements were collected on a single laptop configuration and workload (nginx), and may not generalise to other images, hardware, or larger replica counts.

9. Conclusion and Future Work

This project presented Kubernetes Lite, a single-host container orchestration simulator that reproduces the desired-state reconciliation, dynamic horizontal scaling, real-time monitoring, and automated self-healing behaviours of Kubernetes using only Docker, Flask, and the Docker Python SDK. Controlled experiments showed that the system deploys and scales services within a few seconds per replica and recovers from induced container failures in an average of 7.3 seconds, within the project's 10-second target. While Kubernetes Lite intentionally omits production concerns such as distributed state, multi-node scheduling, and hardened security, these omissions are precisely what allow the core orchestration concepts to be taught and inspected directly, in roughly 300 lines of transparent Python.

Several directions are proposed for future work:

- Implement metric-based autoscaling (an HPA-style controller) that adjusts replica counts from Docker stats API readings rather than requiring manual scaling.
- Introduce a per-deployment load balancer (nginx or HAProxy) to distribute traffic across replicas and demonstrate the Kubernetes Service abstraction.
- Replace the in-memory registry with a lightweight SQLite-backed state store so that desired state survives a controller restart.
- Simulate multiple nodes using Docker-in-Docker or Docker Compose to explore node affinity and failure scenarios.
- Add time-series metrics collection and chart-based visualisation (e.g., Chart.js) of CPU/memory usage and healing frequency.
- Support declarative YAML deployment manifests, introducing students to infrastructure-as-code practices.

References

References are formatted in IEEE style.

- [1] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, and I. Brandic, "Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility," *Future Generation Computer Systems*, vol. 25, no. 6, pp. 599–616, Jun. 2009.
- [2] I. Foster, Y. Zhao, I. Raicu, and S. Lu, "Cloud computing and grid computing 360-degree compared," in *Proc. IEEE Grid Computing Environments Workshop*, Austin, TX, USA, Nov. 2008, pp. 1–10.
- [3] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes: Lessons learned from three container-management systems over a decade," *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016.
- [4] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, article 2, Mar. 2014.
- [5] M. Grinberg, *Flask Web Development: Developing Web Applications with Python*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2018.

- [6] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., Site Reliability Engineering: How Google Runs Production Systems. Sebastopol, CA: O'Reilly Media, 2016.
- [7] S. Newman, Building Microservices: Designing Fine-Grained Systems. Sebastopol, CA: O'Reilly Media, 2015.
- [8] J. Turnbull, The Docker Book: Containerization is the New Virtualization. Brooklyn, NY: James Turnbull, 2014.
- [9] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Special Publication 800-145, Gaithersburg, MD, USA, Sep. 2011.
- [10] Kubernetes Community, "Kubernetes documentation," The Linux Foundation, 2024. [Online]. Available: <https://kubernetes.io/docs/>
- [11] Docker Inc., "Docker documentation," 2024. [Online]. Available: <https://docs.docker.com/>